Solution: Removing Nodes From Linked List

Problem Statement

Write Recursive Approach for Removing Nodes From Linked List.

Given a singly linked list and a target value, write a recursive algorithm to remove all nodes from the linked list that have the target value.

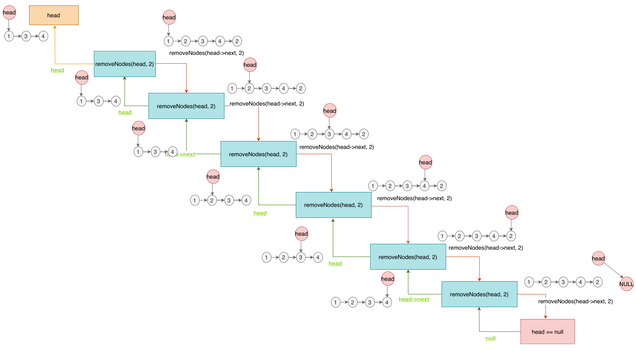
Examples:

Sr #	Input	Expected Output	Description
1	`1 -> 2 -> 3 -> 4 -> 2`	`1 -> 3 -> 4`	Remove all nodes with value `2` from the list.
2	`2 -> 2 -> 2 -> 2 -> 2`	`null`	Remove all nodes with value `2` from the list.
3	`1 -> 3 -> 5 -> 7`	`1 -> 3 -> 5 -> 7`	No nodes with value `2` exist in the list.

Solution:

The algorithm follows a recursive approach to remove nodes with the target value from the linked list. The key steps are as follows:

1. If the linked list is empty, return `null`.
2. Recursively traverse the linked list, moving to the next node.
3. If the current node has the target value, skip it and return the result of the recursive call on the next node.
4. If the current node does not have the target value, set its next pointer to the result of the recursive call on the next node.
5. Return the current node as the new head of the modified linked list.



Code

Here is the code for this algorithm:

Java

```java
// class ListNode {
//     int val;
//     ListNode next;

//     ListNode(int val) {
//         this.val = val;
//     }
// }

public class Solution {

    public static ListNode removeNodes(ListNode head, int target) {
        // Base case: If the list is empty, return null
        if (head == null) {
            return null;
        }

        // Recursively remove nodes with target value from the remaining list
        head.next = removeNodes(head.next, target);

        // Check if the current node has the target value
```

Run Code Result | Stdout

You must run your code first

Execute

Space and Time Complexity

The time complexity of the algorithm is $O(N)$, where n is the number of nodes in the linked list. This is because the algorithm needs to visit each node once to remove nodes with the target value.

The space complexity of the algorithm is $O(N)$, where n is the number of recursive calls on the stack. This is because for each recursive call, additional stack space is required to store the function call context. In the worst case, when all nodes have the target value, the recursion depth is n, resulting in $O(N)$ space complexity.